

Introduction to AI Agents

A lot has been going on with AI over the past few years. Prompt engineering, context, windows, tokens, embeddings,

rag, vector DB, MCPs, agents, lang chain, langgraph, Claude, Gemini, and more. If you felt left out, this is the only video you'll need to watch to catch up. In this video, we assume you know absolutely nothing and try to explain all of these concepts through a single project so that by the end of it, you go from zero to gaining an overall understanding of everything that's going on with AI. We'll start with AI fundamentals, then move on to rag, vector DB, lang chain, langgraph, MCP, prompt engineering, and finally put it all together with a complete system. Let's start with the basics. When you

How LLMs work in real time?

ask an AI model a question, it's typically answered by a subset of AI called large language models. Large language models have gotten popular right around when ChatGPT was released in late 2022 when we started to see language models get larger in size because of their obvious benefits in performance. So let's dig a bit deeper to understand how large language models are able to process requests that we send. Popular LLMs like OpenAI GPT-4, Anthropic's Claude, and Google's Gemini are all transformer models that are trained on large sets of data. The size of training tokens can go up to tens of trillions of tokens that are used to train these models. And the training data includes data from thousands of different domains like healthcare, law, coding, science, and more. But when we work in TechCorp, the 500 GB of data that we have aren't part of the training data that was used to train the model, which means that in order for us to use the LLMs to ask questions about the TechCorp's internal documents, we need the ability to pass in data to the LLM. One of the ways that we can pass the data into the model is by adding them to the conversation history functions like a short-term memory where during the duration of the conversation, all of this context is kept in memory. And this memory is called the context window. Context windows are measured in tokens which is roughly 3/4 of a word for English text. The context window is typically limited in size and the upper limit varies depending on the model. Some models like GPT-4 have 256,000 tokens whereas Anthropic's Claude Opus 4 has 200,000 tokens and Google's Gemini 2.5 Pro has 1 million tokens. So as you can see the total upper bound for how much context can be stored for each model can vary. While the context window plays an important role in storing them

in memory, there are practical limitations in how LLM treats what's inside the context window. For example, if I asked you to memorize the pi digits 3.141592653589791 and asked you to recite it, some of you might have a hard time committing that many numbers all at once, which is similar to how LLM's context window works. So therein lies the current limitations in LLM. How much context can it hold in a given time? This can vary depending on model to model. For example, a lot of nano, mini, and flash models can have very small context windows in the size of 2,000 to 4,000 tokens, which amounts to about 1,500 to 3,000 words. Conversely, bigger models like GPT4.1 and Gemini 2.5 Pro offer context windows up to 1 million tokens, which is equivalent to roughly 7,500 words or 50,000 lines of code. So, as you can see, choosing the right model for the task can be very important. For example, if you downloaded a novel in a txt format and you wanted to change the script, choosing a model that offers a large context window would be best. Conversely, if you are working on a small document and require very low latency, meaning faster responses, using flash and nano variants would be best. Here's another angle to look at when it comes to memory in LLMs. Let's say I ask you this question. Sally and Bob own an apple farm. Sally has 14 apples. Apples are often red. 12 is a nice number. Bob has no red apple, but he has two green apples. Green apples often taste bad. How many apples do they all have? This might require you to think about the problem a little bit to get to the final answer, which is 16. That's because the context here includes information that is completely irrelevant to the question, which is to count how many apples they have in total. The fact that apples are red or green or how it tastes have nothing to do with the total number of apples that they have because they either have the apple or they don't. Now that we have a grasp on what context window provides, Techcorp's 500 GB of documents, this creates an immediate problem. Even the largest context window, like Gemini 2.5 Pro's 1 million tokens, can hold only about 50 files of typical business documents all at once. We need our AI model to understand all 500 gigabytes, but it can only see a tiny fraction at a given moment. This is

Embeddings & Vector Representations

where embedding comes in, and they're absolutely crucial to understand. Embeddings transform the way we think about information. Instead of storing text as words, we convert meaning into numbers. The sentence employee vacation policy and staff time off guidelines use completely different words, but they mean essentially the same thing. Embeddings capture that semantic similarity. And here's how it works. An embedding

model takes a text and converts it into a vector. Typically, 1536 numbers that represent the meaning. Similar concepts end up with similar number patterns like vacation and holiday will have vectors that are mathematically close to each other. For TechCorp, this means that we can find relevant documents based on what someone means, not just the exact word that they've used. When an employee asks, "Can I wear jeans to work?" Our system will find the dress code policy, even if it never mentions the word jeans specifically. Now that we understand how LLMs and embeddings work, we will need a system that ties everything together. In our

How LangChain works?

case, Tech Corp needs a chatbot where customers can ask questions about the company policy, product information, and support issues. The chatbot needs to remember conversation history, access the company knowledge base, and handle complex multi-step interactions. Your first instinct might be to use OpenAI's SDK to build a quick chat interface. But you quickly realize that there are massive missing pieces. Storing chat messages, maintaining conversation context, connecting to Tech Corp's internal knowledge base, and handling the possibility that the company might switch from OpenAI to Anthropic or Google in the future. And now what seemed like a simple project becomes a massive undertaking. While you can write your own implementation to connect them, there's already a well-established abstraction layer called langchain. Langchain is an abstraction layer that helps you build AI agents with minimal code. It addresses all those pain points using pre-built components and standardized interfaces. But first, let's understand the crucial difference between an LLM and an agent. When you use large language models like GPT, Claude and Gemini directly, you're using them as static brain that can answer question based on their training data. An agent on the other hand has autonomy, memory, and tools to perform whatever task it thinks that is necessary to complete your request. For TechCorp's customer support scenario, imagine a customer asks, "What's your company's policy on refunding my product that arrived damaged?" An agent will self-determine how it should answer that request based autonomously instead of traditional software that requires conditional statement that determines how a program should execute. Langchain comes with extensive pre-built components that handle the heavy lifting for TechCorp Chatbot. Langchain chat models provide direct access to LLM

providers. Instead of writing custom API integration code, you can set up OpenAI with open bracket model equals GPT3 turbo. So if the requirements change to use entropic instead, you simply change one line LLM equals chat entropic open bracket model equals claw 3 sonnet. This same pattern applies to every other capability techp needs. Memory management uses memory saver to automatically store and retrieve chat history, which means there's no need to build your own database schema or session management. Vector database integration works through standardized interfaces. Whether you choose Pine Cone or Chroma DB, Langchain provides consistent APIs and we'll go through what a vector database is in the next couple chapters. For text embedding, it uses OpenAI embeddings or similar components to convert Tech Corp's document into vector representation. The embedding process becomes a single function call instead of managing API connections and data transformations manually. Finally, tool integration allows the agent to access external system. So if you need to query Tech Corp's customer database, you can simply create a tool that the agent can call when it determines customer specific information is needed. Without lang chain, you would need to build all of this infrastructure yourself. API management for multiple LLM providers, vector databases, SDKs, embedding pipelines, semantic search logic, state management, memory system, and tool routing. The complexity grows exponentially. Lang chain's component library includes modules like chat anthropic API connections. Chromad vector database operations, OpenAI embeddings for texttovector conversion, memory saver for chat history management, custom tool definitions for external system integrations. The agent orchestrates these components based on the conversation context. So as we're talking about tech corp depending on what the question is asked the agent will now use the given tools like vector databases as well as the context it built from conversation memory and the system prompt written in API layer autonomously handle your request and you can extend the agents abilities beyond this example by using other pre-built tools that lang chain offers like custom database access web search local file system access and more. Now that we covered the conceptual elements of lang chain, let's look at how it looks like on a practical level. We can look over at this lab specifically geared towards how to use lang chain. All right, let's

Practice Labs - Your First AI API Call

start with the labs. In this lab, we're going to explore how to make your very first AI API calls. The mission here is to take you from absolute zero to being able to connect, call, and understand responses from OpenAI's APIs in just a few progressive steps. We begin by verifying our environment. In this step, we're asked to activate the virtual environment. Check that Python is installed. Ensure the OpenAI library is available and confirm that our API keys are set. This is important because without this foundation, nothing else will work. Once the verification runs successfully, the lab will confirm that the environment is ready. Next, we take a moment to understand what OpenAI is. Here we're introduced to the company behind chatbot and their family of AI models including GPT4, GPT4.1 Mini and GPT 3.5. The narration highlights that we'll be working with the Python OpenAI library which acts as a bridge between our code and OpenAI server. With that context set, we move into task one. In this task, we're asked to open up a Python script and complete the missing imports. Specifically, we need to import the OpenAI library and the OS library. After completing these lines, we run the script to make sure that the libraries are properly installed and ready to use. If everything is correct, the program confirms that the import worked. From here, we transition into authentication and client setup. Here, the lab explains the importance of an API client, an API key, and the base URL. The API key works like a password that identifies us and grants access, while the base URL defines the server location where requests are sent. This prepares us for task two. In task two, we open another Python script and are asked to initialize the client by plugging in the correct environment variables. This involves making sure we pass the OpenAI API key and OpenAI API base. Once those values are filled in, we run the script to verify the client has been properly initialized. If done correctly, the script confirms the connection to OpenAI's servers. Once the setup is complete, we move on to the heart of the lab, making an API call. Before jumping into it, we learn what chat completions are. This is OpenAI's conversational API where we send messages and receive messages just like a chat. Lab explains the three roles in a conversation. System, user, and assistant, and how the request format looks like in Python. That takes us into task three. Here we open the script, uncomment the lines that define the model, role, and content, and then configure it. So the AI introduces itself. Once we run the script, if all is correct, the AI should respond back with an introduction. This

is the first live call to the model. Next, we're guided into understanding the structure of the response object.

The lab breaks down the response path, showing how we drill down into the response, choices, message content to

extract the actual text returned by the AI. Although the response object contains other fields like usage,

statistics, and timestamps, most of the time what we really need is the content field.

That brings us to task four,

where we're asked to update the script to extract the AI's response using the exact path. Running the script here

confirms that we can successfully capture and display the text that the AI returns.

Once we've mastered making

calls and extracting responses, the lab shifts gears to tokens and costs. We

learned that tokens are the pieces of text used by the model that every request

consumes tokens. Prompt tokens are

what we send in. Completion tokens are what the AI sends back and total tokens are

the sums of both. Importantly,

output tokens are more expensive than input tokens. So being concise can save

money. Finally, in task five, we're

asked to extract the token usage values, prompt completion, and total tokens from the response. The script is already set

up to calculate costs. So once we complete the extraction and run it, we can see exactly how much the API call

costs. The lab wraps up by congratulating us. At this point, we verified our

environment, connected to

OpenAI, made real API calls, extracted responses, and calculated costs. The key

takeaway is remembering how to navigate the response object with `response.content`.

Some of the finer points and details like exploring usage fields or playing with different models are left for you

to explore yourself. But by now, you should have a solid foundation for working with AI

APIs and be ready for

what comes next in the upcoming labs. All right, let's start with the labs. In this lab,

we're going to explore Lang

Chain and understand how it makes working with multiple AI provider simpler and

faster. The key idea here is

that instead of being locked into one provider's SDK and rewriting code whenever you switch, Langchain offers

one interface that works everywhere. With it, you can move from OpenAI to Google's

Gemini or XAI's Gro by changing

Practice Labs - LangChain

just a single word. We begin the environment verification. In this step, we're asked to

run a script that checks

whether Langchain and its dependencies are installed, validates our API keys and our

base URL, and confirms that we

have access to different model providers. Once this check passes, we're ready to start

experimenting. The first test compares the traditional OpenAI SDK approach with Lang. We have to write 10 or more lines of boilerplate code just to make an API call. If we want to switch to another provider, we'd have to rewrite all of it. With Langchain, the same logic is cut down to just three lines. And switching providers is as simple as changing the model name. In this task, we're asked to complete both versions in a script and then run them side by side.

This is where we really see the 70% reduction in code. The second task demonstrates multimodel support. Here we're asked to configure three providers. Open GPT4, Google's Gemini, and XAS Gro, all with the same class and structure. Once configured, we can run the exact same prompt through all of them and compare their responses. This is especially powerful when you need to do AB test or balance cost because you can evaluate multiple models instantly without changing your code structure. In the third task, we're introduced to prompt templates. Instead of writing separate hard-coded prompts for every variations, we create one reusable template with placeholders. Then we can fill in the variables dynamically just like fstrings in Python. This eliminates the nightmare of maintaining hundreds of slightly different prompt files. After completing the template, we test it with multiple inputs to see how the same structure generates varied responses. The fourth task takes a step further by introducing output parsers. Often AI responses are just free text, but what our code really needs are structured objects. Here we're asked to add parsers that can transform responses into lists or JSON objects. In this way, instead of dealing with unstructured sentences, we can access clean Python lists or dictionaries that our application can use directly. Finally, we reach task five, which is all about chain composition. Langchain allows us to connect components together with pipe operator. Just like Unix pipes, instead of writing multiple variables for each step, creating a prompt, sending it to the model, getting a response, and parsing the result, we simply chain everything together. With one line, we can link prompts, models, and parsers, and then invoke the chain to get the structure output. It's a much cleaner and more scalable way to build AI pipelines. By the end of this lab, we've learned how a lane chain reduces boiler plate, enables multimodel flexibility, creates reusable templates, parses structured outputs, and ties everything together with elegant chaining. Some of the finer details like experimenting with more complex parser setups or chaining additional steps are left for you to explore on your own.

Prompt Engineering Techniques

Now, we come to the technique, but it's not a technique in the sense of building a chain application like we just did.

No, we're talking about a technique that involves how you send your prompt to the agent that we just built. In other

words, prompt engineering. When you send a prompt as an input to TechCorp's AI document assistant that we just built, the quality of your prompt directly impacts the quality of responses you

receive. While AI agents can certainly handle a wide range of prompts, understanding prompt techniques help you

communicate more effectively with TechCorp's system. For example, if you prompt the agent with this question,

"What is the policy?" It can pull a lot of details that are irrelevant. Sending a more specific prompt like, "What's the

company's remote work policy for international employees?" will lead to a more accurate result from the agent. And

the same thing applies to role definition when you're describing the role of the agent. For example, you

might descriptively write out a detailed prompt like, "You are a tech customer support expert." When you are asked

about the company's policy, you are to always respond with bullet points for easier readability. As you can see,

being able to control the agent's behavior can directly benefit from a well-written prompt. This type of

technique is referred to as prompt engineering. And there are different prompt techniques like zero-shot, one-shot,

few-shot, and chain of thought prompting have their own use case for the task. For example, zero-shot prompting means that

we are asking AI to perform a task without providing any examples. So if

you send a prompt, write a data privacy policy for our European customers, you're essentially relying entirely on

the AI's existing knowledge base to write the data policy document. Since within the prompt, we're not giving any

examples of what they are. One-shot and few-shot prompting is similar to zero-shot but in this case we're

providing examples of how the agent should respond directly within the prompt. For example, you might say

here's how we format our policy documents. Now write a data privacy policy following the same structure

because you provided a template. The AI follows your specific formatting and style preferences more consistently. And

conversely, fine-tuning is the act of learning from the LLM side where even though the LLM might not have seen the

exact training data for how to process your unique request, it's able to demonstrate the ability to fulfill your

request from similar examples provided. And finally, chain of thought prompting is a style of prompting where you provide the model with a trail of steps to think through how to solve specific problems. For example, instead of prompting the AI agent with fix our data retention policy, you might instead use chain of thought prompting to say here's how you fix data retention policy. Review current GDPR requirements for data retention periods. Then analyze our existing policy for specific gaps. Then research industry best practices for similar companies. And finally, draft specific recommendations within implementation steps. Now fix our customer policy. As you can see, providing how LLM should go through in breaking down a specific request for how data retention policy should be fixed gives an exact blueprint for how the LLM should then go and fix the customer policy, which in this case, we're not explicitly telling the agent how to fix the policy for that, but it gives the reasoning steps for the model to fix accordingly. So, in this lab, we're

Practice Labs - Master Prompt Engineering

going to master prompt engineering using lang chain. The main problem being addressed here is that AI can sometimes give vague or inconsistent responses or not follow instructions properly. The solution is to use structured prompting techniques, zero shot, one shot, viewshot, and chain of thought. Each of which controls the AI's behavior in a different way. We begin by verifying the environment. The provided script checks that lang chain and its OpenAI integrations are installed, confirms that the API key and base URL are set and ensures that prompt template utilities are available. Once this verification passes, we're ready to move into tasks. The first task introduces zero prompting. In this exercise, we're asked to compare what happens when we provide a vague instruction versus when we write a very specific prompt. For example, simply asking the AI to write a policy results in a long generic essay. But when we specify write a 200word GDPR compliant privacy policy for European customer with 30-day retention period, the response is focused, useful, and aligned to the constraints. This demonstrates why being specific is crucial in zero shot prompts. The second task moves us to oneot prompting. Here we provide one example for the AI to follow almost like showing a single template. For example, if we gave the AI one refund policy example with five structured sections, we can then ask it to produce a remote work policy and it will replicate the same style in structure. This shows how one example can set the tone and ensure consistency

across many outputs. Next, in task three, we expand on this with few shot prompting. Instead of one example, we provide multiple examples so the AI can learn not only the format but also the tone, patterns, and style. For example, giving three examples of emphatic support replies teaches the model how to handle customer support issues consistently. Once the examples are in place, the AI can generate new responses that follow the same tone and structure, making it especially powerful for use cases like customer service. In task 4, we're introduced to chain of thought prompting. This technique encourages the AI to show its reasoning step by step. Instead of vague online answer, the AI breaks a problem into steps and works through it systematically. This results in clear or more reliable and more accurate outputs, particularly for complex reasoning tasks. Finally, task 5 brings all of these techniques together in a head-to-head comparison. We run the same problem through zero shot, one shot, few shot and chain of thought prompts to see the difference. Each approach has its strength. Zero shot is quick. One shot ensures formatting. Few shot enforces tone and consistency and chain of thought excels at detailed reasoning. The outcome shows that choosing the right technique can dramatically improve results depending on the task. By end of this lab, we not only learned what each prompt method is, but we've also seen them in action. The key takeaway is that the right technique can make your prompts 10 times more effective. Some of these exercises are left for you to explore and refine. But now you have the foundation to decide whether you need speed, structure, style, or reasoning in your AI responses. That wraps up this narration. And with it, you're now ready to move on to the next lab onto vector databases and semantic search.

Let's do a quick recap of what we just built. We learned about what LLM is and how LLMs use what's inside the context

Vector Databases Deep Dive

window. After learning about LLMs, we wanted to solve Tech Corp's business requirements of searching for 500 GB worth of data. In order to do that, we determined that embedding is a good way to search a massive set of documents. After that, we went over Langchain and what functions they serve, which is that they allow us to easily build Gentic application like Tech Corps chatbot. So now that we have the lang chain application, we need to be able to search through these large sets of documents. Let's say inside the 500 GB of documents, your company has a document called employee handbook that covers policies like time off, dress code, and

equipment use. Employees might ask terms like vacation policy, but miss time off guidelines. While these are common questions that people would typically ask, building a database around this requirement can be tricky. In a conventional approach where data is stored in a structured database like SQL, you typically need to do some amount of similarity search like select all from documents content like vacation or vacation policy with a wild card before and after to look for details about questions on holiday. To expand your result set, you might increase the scope by adding VA or VAC space P. However, the drawback to this approach is that it puts the onus on the person searching for the data to get the search term formatted correctly. But what if there was a different way to store the data? What if instead of storing them by the value, we store the meaning of those words? This way, when you search the database by sending the question itself of can I request time off on a holiday based on the meaning of those words contained in the question, the database returns only relevant data back. This is a spirit of what vector databases tries to address storing data by the embedding. So essentially instead of searching by value, we can now search by meaning. Popular implementation of vector databases include pine cone and chroma. These platforms are designed to handle embeddings at scale and provide efficient retrieval based on semantic similarity. And these are also great use cases for prototyping something quick. While conceptually this seems straightforward, there's a bit of an overhead in setting this up. And you might be asking, well, can we just throw the employee handbook into the database like we just did for SQL database? Not quite. And here's why. With SQL database, the burden is put on the user searching to structure the database. But with vector databases, the burden is put on you who is setting up the database since you are trying to make it easier for someone searching for the data. And you can imagine why a method like this is becoming extremely popular when paired with large language models in AI since you don't have to train separately on how LLM should search your database. Instead, the LLM can freely search based on meaning and have the confidence that your database will return relevant data it needs. So let's explore some of the key concepts behind what goes into setting up a basic vector database. Let's start with embedding. Embedding is really the key concept that makes the medium go from value to meaning. In SQL, we store the values contained in the employee handbook as a straightup value. But in a vector database, you need to do some extra work up front to convert the

value into semantic meanings. And these meanings are stored in what's called embeddings. For example, the words holiday and vacation should semantically share a similar space since the meaning of those words are close to each other. So before the sentence employee shall not request time off on holidays in the document is added to the database. The system runs through an embedding model and the embedding model converts that sentence into a long vector of numbers and when you search the database you are actually comparing this exact vector. That way when someone later asks can I take vacation during a holiday even though the phrasing is a little bit different the database can still service the request. And this is the fundamental shift. Instead of searching by exact wording, we're now searching by meaning. Another important concept is dimensionality. And you might be asking, why do I have to worry about dimensionality? Can't I just throw the words into embedding and store it into the database? There's one more aspect in embedding that you need to think about, and that's dimensionality. Typically, a word doesn't just have one meaning to learn from. For example, the word vacation can have different semantics depending on the context that is used in. and capturing all those intricacies like tone, formality, and other features can give richness to those words. Typically, dimensions we use today are 1536 dimensions, which is a good mix of not having too much burden in size, but also giving enough context to allow for depth in each search. Once the embedding is stored with proper dimension, there are two other major angles that we need to consider when we're working with vector databases. And this is the retrieval side. Meaning now that we store the meaning of those words, we have to take on the burden of the retrieval side of embeddings. Since we are not doing searches like we did in SQL with a wear query, we need to make a decision on what would technically be counted as a much and by how much. This is done by looking at scoring and chunk overlap. And if you're at this point wondering, this seems like a lot of tweaking just to use vector database. And that's the serious trade-off you ought to consider when using a vector database, which is that while a properly set up vector database makes searching so much more flexible, getting the vector database properly configured often adds complexity up front. So with that in mind, scoring is a threshold you set to how similar the results need to be to be considered a proper match. For example, the word Florida might have some similarity to the word vacation since it's often where people go for vacation. But

asking the question, can I take my company laptop to Florida is very different than does my company allow vacation to Florida? Since one is asking about a policy in IT jurisdiction and the other is about vacation policy. So setting up a score threshold based on the question can help you limit those low similarities to count as a match. Okay, there's one final angle which is chunk overlap. So in SQL, we're used to storing things rowby row, but in vector databases, things look a little bit different. When we're storing values in vector databases, they're often chunked going into the database. So when we chunk down an entire employee handbook into chunks, it's possible that the meaning gets chunked with it. That's why we allow chunk overlap so that the context spills over to leave enough margin for the search to work properly. In this app, we're going to build a semantic search engine step by step. The story begins with TechDoc Inc. where

Practice Labs - Build Semantic Search Engine

users search through documentation 10,000 times a day. But more than half of those searches fail. Why? Because traditional keyword search can't connect reset password with password recovery process. Our mission is to fix that by building a search system that understands meaning, not just words. We begin with the environment setup. In this step, we're asked to install the libraries that make vector search possible. Sentence transformers for embeddings. lang chain for orchestration, chromadb for vector database and few utility libraries like numpy. Once installed, we verify the setup using a provided script. If everything checks out, we're ready to move forward. Next, we take a moment to understand embeddings. These are the backbone of semantic search. Instead of treating texts as words, embedding converts text into numerical vectors. Similar meanings end up close to each other in this mathematical space. That means forgot my password in account recovery looks very different in words but almost identical in vector 4. This is a magic that allows our search engine to succeed where keyword search fails. That takes us into task number one where we put embedding into action. We open the script, initialize the mini LM model, encode both queries and documents and then calculate similarity using cosine similarity. Running the script demonstrates how a search for forgot password successfully matches password recovery, showing semantic understanding in real time. Once we understand embeddings, we move to document chunking. Large documents can't be embedded all at once. So, we need to split them

into smaller chunks, but if we cut too bluntly, we lose context. That's why overlapping chunks are important. They preserve meanings across boundaries. For example, setting a chunk size of 500 characters with 100 characters overlap can improve retrieval accuracy by almost 40%. Lang chain helps us do this intelligently. In task number two, we put this into practice by editing a script to import lang chain's recursive characters text splitter and set the chunking parameters. Running the script confirms that our documents are now split into overlapping pieces ready for storage. The next concept we explore is vector stores. Embeddings are just numbers. We need a system to store and search through them efficiently. That's where Chroma comes in. It's a production ready vector database that can handle millions of embeddings, perform similarity search in milliseconds, and support metadata filtering. In task number three, we're asked to create a vector store using Chroma DB. We import the necessary classes, configure the embedding model, and then run the script. Once confirmed, we have a working vector store that can accept documents and retrieve them semantically. Finally, we bring everything together with semantic search. Here we implement the full pipeline, convert the user query into an embedding, search the Chroma store, retrieve the most relevant document chunks, and return them to the user. For example, a query like work from home policy will now correctly surface remote work guidelines. In task 4, we configure the query, set the number of top results to return, and establish a threshold for similarity scores. Running the script validates our search engine end to end. The lab closes with a recap. We started with a broken keyword search system where 60% of the search failed. Along the way, we learned about embeddings, smart document chunking, vector stores, and semantic search. By the end, we built a production ready search engine with 95% success rate. Some of the deeper experiments like adjusting chunk sizes, testing different embedding models and or adding metadata filters are left for you to explore on your own. So is it possible that instead of searching through the entire 500 GB of documents, AI assistant can find them?

RAG (Retrieval Augmented Generation)

into their context window and generate output. This is called rag or retrieval augmented generation. Let's say your company used the AI assistant to ask this question. What's our remote work policy for international employees? In order to understand how rag works, we need to break them into three simple steps.

Retrieval, augmented, and generation. Starting with retrieval, just like how we convert the document into vector embeddings to store them inside the database, we do the exact same step for the question that reads, "What's our remote work policy for international employees?" Once the word embedding for this question is generated, the embedding for that question is compared against embeddings of the documents. This type of search is called semantic search where instead of searching by the static keywords to find relevant contents, the meaning and the context of the query is used to match against the existing data set. Moving on to augmentation in rag refers to the process where the retrieved data is injected into the prompt at runtime. And you might think why is this all that special? Typically, AI assistants rely on what they learned during pre-training, which is static knowledge that can become outdated. Instead, our goal here is to have the AI assistant rely on up-to-date information stored in the vector database. In the case of RAG, the semantic search result feeds into the prompt that essentially serves as an augmented knowledge. So, for your company, the AI assistant is given details from company's documents that are real, up-to-date, and private data set. And all this can occur without needing to fine-tune or modify the large language model with custom data. The final step of rag is generation. This is a step where AI assistant generates the response given the semantic relevant data retrieved from the vector database. So the initial prompt that says what's our remote work policy for international employees? The AI assistant will now demonstrate its understanding of your company's knowledge base by using the documents that relate to remote work and policy. And since the initial prompt specifies a criteria of international employees, the generation step will use its own reasoning to wrestle with the data provided to best answer the question. Now, RAG is a very powerful system that can instantly improve the depth of knowledge beyond its training data. But just like any other system, learning how to calibrate is an acquired skill that needs to be learned to get the best results. Setting up a rag system will look different from one system to another because it heavily depends on the data set that you're trying to store. For example, legal documents will require different chunking strategies than a customer support transcript document. This is because legal documents often have long structured paragraph that need to be preserved and intact. While

conversational transcript can be just fine with sentence level chunking with high overlap to preserve context. In

Practice Labs - RAG Implementation

this lab, we're taking our semantic search system to the next level by adding AI power generation. Up until

now, we've been able to find relevant documents with high accuracy. For example, matching remote work policy

when someone searches work from home. But the CEO wants more. Instead of retrieving a document, the system should

actually answer the user's questions directly. something like yes, you can work three days from home, not just

showing a PDF. We begin the environment setup. In this step, we're asked to activate the Python environment and

install the key libraries. These include Chroma DB for vector storage, sentence transformers for embeddings within lane

chain with integrations for OpenAI and hugging face. Once installed, we verify everything using the provided script to

ensure the rack framework is ready. Next, we move into task number one, setting up the vector store. Here we

initialize a chromabb client. Create our get collection named tech corp brag and configure the embedding model all mini6v2. This is where we get our system of

memory. A place where all of our company documents will be stored as vectors so that we can search them semantically. In

task number two, the focus shifts to document processing and chunking. Unlike our earlier lab where we split text into

fixed-size character chunks, here we upgrade to paragraph-based chunking with smart overlaps. The goal is to preserve

meaning so that each chunk contains complete thoughts. This is crucial for RAG because when AI generates answers,

the quality depends on having coherent chunks of context. From there, we go to task number three,

LLM integration. This is where we connect OpenA model GPD4.1 Mini. The API key and base are already preconfigured for us. We just need to set the generation

parameters like temperature, max tokens, and top P values. Once integrated, we can test simple text generation before layering on retrieval

and augmented steps. Task number four introduces prompt engineering for rag. We're asked to build a structured prompt

template that always ensures context is included. The system prompt makes it clear that answers must come only from

the retrieved documents. If the information isn't in context, the AI must respond with, "I don't have that

information in the provided documents." This keeps our answers factual and prevents hallucinations. Finally, we

reach task number five, the complete rag pipeline. Here we wire everything

together. The flow is embed the user query, search Chromad, retrieve the top three chunks, build a context-aware prompt, and generate an answer using LLM. The final touch is a source attribution. Every answer points back to the document it was derived from. This transforms the system into a full production-ready Q&A engine. At the end, we celebrate RAG mastery. What started as simple document search has evolved into a powerful system that retrieves, augments, and generates answers. This is the same architecture that powers tools like Catchup, Claude, and Gemini. Some parts like experimenting with different chunk strategies, refining prompts are left for you to explore yourself. But by now you've built a complete RAG system that answers questions with context, accuracy, and confidence. Now that we covered the conceptual elements of RAG, let's look at how it looks on a practical level. To better understand this, we can look over at this lab specifically geared towards how to use RAG. Now we covered the basic concepts of simple chat application that allows us to chat with documents using vector databases and RAG. Most business cases in the real world may be slightly more complicated. For example, in Tech Corp's case, the business requirement might extend to more complex requirements like being able to connect the agent to human resource management system to pull employee documents to cross-reference and make personalized responses. However, LangChain has limitations. When business requirements

LangGraph for AI Workflows

become more complex like multi-step workflows, conditional branching or iterative processes, you need something more sophisticated for better orchestration. That's where LangGraph becomes essential. LangGraph extends LangChain to handle more complex multi-step workflows that go beyond simple question and answer interactions. For example, if a customer asks, I need to understand our data privacy policy for EU customers. Since we assume that inside the 500 GB of database, it contains details about EU-specific regulations, we need to create a system that can analyze Tech Corp's data privacy policies for EU customers, ensuring compliance with GDPR, local regulations, and company standards. While in a traditional software development, you need to write code that can sequentially and conditionally call different sections of the code to process this request. With LangGraph, this becomes a graph where each node handles a specific responsibility. For example, node one, search and gather privacy policy documents. Node two, extract and clean document content. Node three, evaluate

GDPR compliance using LLM analysis. Node four, cross reference the local EU regulations. And node five, identify compliance gaps and generate recommendation. A node is an individual unit of computation. So think of a function that you can call. Once you have all the nodes created in langraph, you will then need to connect them and this connection is called an edge. Edges in langraph define execution flow. For example, after node one gathers documents, the edge routes to node two for content extraction. And after node 3 evaluates compliance, a conditional edge either routes to node four for additional analysis or jumps to node five for report generation. And one final concept to keep in mind beyond nodes and edges is shared state between each node. This is possible by using state graph that essentially stores information throughout the entire workflow. For example, class compliance state type dictionary topic string documents list of string current documents optional string compliance score optional integer gaps list of string recommendation list of string can be used for nodes we identified before. As the workflow progresses, each node updates relevant state variables. Node one populates documents with found policy files. Node two processes individual documents and updates current document. Node 3 calculates compliance score. Node 4 identifies gaps. Node 5 generates recommendation. The state graph orchestrates execution based on configured flow. If node 3 determines compliance score is below 75% the conditional edge routes back to node one to gather additional documents. If the score exceeds 75% execution proceeds to node 5 for final report generation. As you can see this creates powerful capabilities loops for iterative analysis conditional branching on intermediate results persistent state that maintains context across the entire workflow. So for Tech Corps compliance assistant, Langraph is an essential tool for workflow automation. All right, let's start with the labs. In this lab, we're diving into Langraph, a framework designed for building stateful multi-step AI workflows. Unlike simple chains, Langraph gives us specific control over how data moves, letting us create branching logic, loops, and decision points. By the end of this journey, we'll have built a complete research assistant that can use multiple tools intelligently. We begin with environment setup. In this setup, we

Practice Labs - Build Stateful AI Workflow

activate the Python virtual environment and install the required libraries. Langraph itself, Langchain, and OpenAI

integration. Once everything is installed, we run a verification script to make sure our setup is ready. With the environment ready, we start small. Task number one introduces us to essential import. We bring in state graph end and type dict to define the data that flows through the workflow. Then we add a simple state field for messages. This is the foundation. State graph holds the workflow and marks completion and the state holds shared data. In task number two, we create our first nodes. Nodes are just Python functions that take state as inputs and return partial updates. In this case, we define a greeting node and an enhancement node. Once connected, one node outputs a basic greeting and the next node improves it with a bit of a flare. This demonstrates how state accumulates step by step. Pass number three is about edges. The connections between nodes. Here we use add nodes and add edges to wire greeting nodes to the enhancement node. With that, we built our first mini workflow. Data flows from one function to another. The state updates along the way. In task number four, we take it further with a multi-step flow. We add new nodes like a draft step and a review step. Connect them and see how the data moves through multiple stages. Each step preserves states, adds detail and passes it on. This mimics real world pipelines where content is outlined, drafted and polished. Pass number five introduces conditional routing. Instead of a fixed flow, the system now decides dynamically. For example, if the query is short, it routes one way. If detailed, it routes another. The router inspects the state and returns the next node name, making workflows flexible and adaptive. Then comes task number six, tool integration. Here we add a calculator tool. The router checks if the query is math related. If so, it routes to the calculator node which computes the answer. This is our first glimpse at how Langgraph lets us integrate specialized tools directly into workflows. Finally, task number seven puts everything together into a research agent. We combine the calculator with a web search tool like duck.go. Depending on the query, the system decides whether to perform a calculation, run a web search, or handle a text normally. This is a dynamic tool orchestration, the foundation of modern AI agents. By the end of this lab, we've gone from simple imports to a fully functional research assistant. We've seen how to build nodes, connect them, design multi-step flows, and add routing logic and integrate tools. Some of the deeper experiments like chaining more advanced tools or refining the router logic are left for you to explore on your own.

Model Context Protocol (MCP)

Now that we covered lang chain and langgraph and understood how techp business requirements can be met by leveraging pre-built tools that it offers, there's one final piece that's been popular since Anthropic released back in November 2022 called MCP or model context protocol. Techorps AI document assistant is working well for internal knowledge base, but employees might now need to access external systems like customer database, support systems, inventory management software, and other thirdparty APIs. And writing custom integrations to all these API connections will take a huge amount of time. MCP functions like an API, but with crucial differences that make it perfect for AI agents. Traditional APIs expose endpoints that require you to understand implementation details leading to rigid integrations tied to specific systems. MCP doesn't just expose tools. It provides self-describing interfaces that AI agents can understand and use autonomously. The key advantage here is that unlike traditional APIs, MCP puts the burden on the AI agent rather than the developer. So when you start an MCP server, an instance starts and establish a connection with your AI agent. For example, the Techorps document assistant might easily have these MCP servers to enable powerful integration. Let's say customer database MCP. When someone asks, "What's the status of the order 1 2 3 4?" The AI uses an MCP to query TechCorb's order management system, retrieves the current status, and provides a complete response. The same logic applies for support tickets, inventory databases and notification system mentioned earlier where we can simply plug into existing integrations of MCP servers to allow the agent to now extend its capabilities. For example, we can create a very simple MCP server code that looks something like this. Here we have fast MCP customer DB that starts the MCP server with the name customer DB at MCB tool that exposes a function to MCP clients. the AI can call like an API function parameters and return types that tells the MCP client what inputs are required and what type of output to expect. Finally, customers variable that is a fake database in this case stored in memory, but in your company's case, you can connect this to a SQL database or MongoDB or any other custom database you might hold customer information on. Now, looking at this code might confuse you on what MCP really is, since it's typically being talked about as a simple plugand play, and you're right to think that way. The difference here is that this MCP server code that's written only needs to be written once, and it doesn't necessarily have to be you. In other words, a

community of MCP developers might have written custom MCP servers for other popular tools like GitHub, GitLab, or SQL databases, and you can simply use them directly on your agent without having to write the code yourself. That's where the power of MCP really comes from. In this lab, we're going deeper into MCP, model context

Practice Labs - Advanced MCP Concepts

protocol, and learning how to extend Langgraph with external tools. Think of MCP as a universal port like USB that

allows AI system to connect to any tool, database or API in a standardized way.

With it, our langraph agents can go beyond built-in logic and integrate external services seamlessly. We begin

with the environment step. In this step, we activate our virtual environment and install the key packages. Lang graph for

the workflow framework, Langchain for the core abstractions, and Langchain OpenAI for the model integration. The

setup also prepares for fast MCP, the framework we'll use to build MCP servers. Once installed, we verify by

learning the provided script, ensuring everything is ready for MCP development. Next, we get a conceptual overview of

the MCP architecture. Here the lab explains that the MCP protocol acts as a bridge between an AI assistant built with Langraph and external tools. The

flow works like this. The MCP server exposes tools and schemas. Langraph integrates with them and queries are

routed intelligently. The naming convention MCP server tool ensures

clarity when multiple tools are involved. A helpful analogy is comparing MCP to USB devices. A protocol is a

port. The server is a device. The tools are its functions. And Langraph is a computer that uses them. That brings us

to task number one, MCP basics. Here we're asked to create our very first MCP server. The task involves initializing a server called calculator, defining a function as a

tool with at MCP.tool

decorator and running it with the SCDIO transport. This shows how simple it is to expose a structured function as an

external tool. that langraph can later consume. In task number two, we integrate MCP with langraph. The

challenge here is to connect the calculator server to an agent. This involves configuring the client fetching

tools from the server create react agent that can decide when to call the calculator selected when needed. Next,

task number three scales things up with multiple MCP servers. Instead of just a calculator, we add another server, in

this case, a weather service. Now, Langraph orchestrates between both. The system retrieves available tools,

creates an agent with access to both servers, and intelligently routes queries. If a user

asks a math question, the calculator responds. If they ask about the weather, the weather tool responds. This is where we see the true power of MCP. Multiple servers are working together under a unified AI agent. The lab wraps up by celebrating MCP mastery. By now, we've created MCP servers, integrated them with ElangRaph, and orchestrated multiple tools. The key takeaways are that MCP is universal. It can connect any tool to any AI. Routing is what gives it power. The design is extendable, so we can add servers anytime. Some deeper explorations like exposing databases, APIs or file systems through MCP are left for you to explore on your own. That concludes this narration. Next, we'll continue the journey by experimenting with resource exposure, human in the loop approval flows, and eventually deploying production ready MCP packages. Now that we have put all these pieces together like context windows, vector

Conclusion

databases, lang chain, langraph, MCP, and prompt engineering, Techcorp is now able to do complex document search that went from manual searching that could have taken up to 30 minutes to now less than 30 seconds using our AI agent. And we also have a higher accuracy using contextaware semantic search like using rag. And finally, the chat application UI allows users to have more satisfaction in working with a tool that can help keep track of conversation history and better intuition overall. And the availability for this is 24/7 as long as the application is running. And this is just the beginning. Imagine layering on predictive analytics, proactive compliance agents, and workflow automation that doesn't just answer questions, but actively solves problems before employees can even ask. The shift from static documents to living intelligent system marks a turning point not just for Tech Corp, but for how every other business can unlock a full value of its knowledge using agents.